

Practical Optimizing UAV Trajectory in Wireless Charging Networks: An Approximated Approach

Yundi Wang[✉], Xiaoyu Wang[✉], He Huang[✉], *Senior Member, IEEE*, and Haipeng Dai[✉], *Senior Member, IEEE*

Abstract—Unmanned Aerial Vehicles (UAVs) can be easily deployed as auxiliary base stations due to their convenience and flexibility. However, limited battery capacity becomes a bottleneck. Promising wireless power transfer (WPT) technologies can provide a continuous power supply for UAVs. Many of the recent works treat the UAV battery capacity as a constraint, which hinders the assurance of continuous UAV operation. Furthermore, most studies employ intelligent path-planning algorithms that lack explicit performance guarantees. In this paper, we study the problem of Practical Optimizing UAV Trajectory in Wireless Charging Networks (POTWCN), which involves planning the trajectory of the wireless-powered UAV in the practical environment with obstacles by selecting candidate passing positions and determining the access order in the charging network. The goal is to maximize the benefit, i.e., balancing the total task completion time and the number of charging stations visited, so as to minimize path length and flight time, and ensure energy constraints with performance bound. To solve this problem, we first formalize the problem and prove its submodularity. Then, we propose the obstacle-aware weighted graph generation algorithm (OWGGA) to deal with the obstacles in the environment, which forms an obstacle-avoidance path using tangents and arcs between two hovering positions and the blocking obstacles. Next, we propose a dynamic charging station selection algorithm (ACSA), which maximizes the UAV's energy utilization by limiting the number of charging stations that can be included. In the algorithm, we introduce the Christofides algorithm and use the path length calculated by OWGGA as the edge weights of the graph. Subsequently, considering the UAV's energy constraints, we iteratively solve the UAV trajectory planning problem by adding the charging station with a maximized marginal benefit to the path. We prove that the proposed algorithm achieves an approximation ratio $1 - 1/e$ as well as the path length is at most $3\pi/4$ times the optimal solution. Simulation results show that our algorithm reduces the flight distance by 38.01% and the task completion time by 34.00% on average.

Index Terms—UAV trajectory optimization, approximation algorithm, wireless charging network.

Received 20 February 2025; revised 30 May 2025; accepted 3 July 2025. Date of publication 7 July 2025; date of current version 3 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62202321 and Grant 62332013 and in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 22KJB520033. Recommended for acceptance by Z. Sun. (*Corresponding author: Xiaoyu Wang.*)

Yundi Wang, Xiaoyu Wang, and He Huang are with the School of Computer Science and Technology, Soochow University, Suzhou 215006, China (e-mail: ydwang330@stu.suda.edu.cn; xywang21@suda.edu.cn; huangh@suda.edu.cn).

Haipeng Dai is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: haipengdai@nju.edu.cn). Digital Object Identifier 10.1109/TMC.2025.3586457

I. INTRODUCTION

UNMANNED Aerial Vehicles (UAVs) have garnered significant attention as a key technology due to their high flexibility, low cost, strong line-of-sight (LoS) communication for both downlink and uplink transmissions, and easy deployment [1]. The future market for UAVs is substantial. According to market research, the overall UAV market is projected to grow to \$48.5 billion by 2029, with a compound annual growth rate (CAGR) of 9.9% [2].

Currently, UAVs can be used as communication base stations for data transmission tasks, as well as they can amplify Internet signals to provide access services to remote areas. However, as the demand for UAV capabilities grows, the energy constraints of UAVs increasingly fail to meet these demands. The rapidly developing technology of Wireless Power Transfer (WPT) offers a potential solution to this issue. For instance, SF Express began piloting drone logistics in 2017, primarily for express delivery in remote areas such as mountains and islands with limited transportation access. SF's drones are equipped with high-capacity batteries and communication modules, enabling long-distance flights and real-time data transmission to assist in communication [3]. PowerLight Technologies has proposed deploying fiber optic power delivery technology on the ground and in the air, capable of transmitting hundreds of watts over hundreds of meters to charge UAVs [4]. Companies such as Huawei and Xiaomi have also introduced the concept of providing wireless charging through 5G base stations [5]. These applications demonstrate that integrating WPT technology with UAVs can effectively enhance UAV endurance, significantly improve task performance, and showcase substantial application potential.

Some research has integrated WPT technology. For example, research has incorporated UAV energy constraints into trajectory planning [6], [7], [8], set up UAV wireless charging stations [9], [10], and considered recharging UAVs during mission execution [11], [12]. However, there are still notable shortcomings. Certain studies perceive the energy constraint as a given condition, neglecting the aspect of recharging. Others consider charging as a static condition without incorporating dynamic planning for UAV trajectories. Still, some utilize intelligent algorithms such as heuristics, without establishing performance bounds.

At the same time, in the path planning problem, it is also necessary to consider real-world environmental factors and model this complex environment. UAVs need to accurately evaluate paths by taking into account fundamental constraints such as flight

time, path length, and energy limitations, and plan reasonable routes to avoid obstacles [13]. Nowadays, UAV path planning has been proven to be an NP-hard problem [14]. However, while path planning primarily focuses on feasibility, our objective is to explore path optimization based on feasible paths, which is a more complex problem.

Specifically, we consider the problem of Practical Optimizing UAV Trajectory in Wireless Charging Networks (POTWCN), i.e., given all environmental information, planning an optimal closed and obstacle-avoiding trajectory for the UAV by selecting candidate passing positions and determining the access order, ensuring all devices are covered without depleting the UAV's battery. The goal is to maximize the benefit, that is, to balance the total task completion time and the number of charging stations visited, while minimizing path length and time as much as possible, and ensuring energy constraints are met.

This problem is challenging for several reasons. First, due to the presence of obstacles in the environment, appropriate obstacle avoidance methods must be employed to ensure that the trajectory calculated during subsequent path planning remains feasible. Second, we need to compute a closed trajectory that passes through all known data collection points while minimizing the cost. This is essentially a Traveling Salesman Problem (TSP), which is NP-hard. Third, we must ensure that the UAV can remain in service, i.e., meeting energy constraints. We need to effectively select and add optimal charging stations to the trajectory to balance total time and the number of selected charging stations. Fourth, we need to consider all factors, including UAV flying distance, energy constraints, etc., to come up with the final algorithm with performance bound.

For the first challenge, we propose an obstacle-aware weighted graph generation algorithm (OWGGA) with $\pi/2$ approximation, which specifically avoids obstacles by using tangent lines from the starting point and the arcs. For the second challenge, we introduce the Christofides algorithm [15] in our algorithm to solve the Traveling Salesman Problem (TSP) and integrate the OWGGA algorithm into the input graph of the TSP algorithm, enabling the calculated trajectory to achieve obstacle avoidance. For the third challenge, we first prove the submodularity of the objective function and then apply a greedy approach. Finally, in the algorithm, we dynamically adjust the number of charging stations the UAV passes through based on its energy level in each iteration. This ensures that the UAV meets energy constraints. We form the final algorithm as the adaptive charging station algorithm (ACSA), which has a $1 - 1/e$ approximation ratio and the path length is at most $3\pi/4$ times the optimal solution.

Simulation results demonstrate that our proposed algorithm reduces the flight distance by 38.01% and requires 34.00% less completion time on average compared with other algorithms, while also ensuring the convergence and stability of the results.

II. RELATED WORK

Trajectory planning for energy-constrained UAVs: For example, Dai et al. [7] addressed UAV-assisted vehicle task offloading to minimize delays. Xu et al. [8] designed a multi-UAV base

station cooperative edge computing scenario with energy constraints. Gong et al. [16] investigated the problem of minimizing both the number of energy-constrained UAV base stations and their energy consumption.

In addition to the aforementioned heuristic algorithms, some related works have also proposed approximation algorithms with provable approximation ratios. For example, Wang et al. [17] present a $1 - (1 - 1/K)^K$ -approximation algorithm to dynamically determine cooperative path planning for K UAVs, optimizing coverage of users' spatio-temporally distributed demands. Li et al. [18] proposed a $1/3$ -approximation algorithm for data collection maximization problems to deal with full data collection from IoT devices. Qu et al. [19] designed a $1/(4 + \epsilon)$ -approximation algorithm (where ϵ is an arbitrarily small positive parameter) for urban sensing tasks using delivery drones, jointly optimizing route selection, sensing time, and delivery weight allocation to maximize delivery and sensing utility under energy constraints. Xu et al. [20] proposed a heterogeneous drone scheduling algorithm with a $1/3$ approximation ratio, coordinating drones with different functionalities to monitor points of interest in disaster scenarios, achieving maximum monitoring revenue under energy constraints. Xu et al. [21] further developed a $1/(6 + \epsilon)$ -approximation algorithm to solve the problem of finding a data collection trajectory for an energy-constrained UAV, maximizing the cumulative utility of collected data, where ϵ is a given constant with $\epsilon > 0$. These studies demonstrate the broad applicability and efficiency of approximation algorithms in UAV trajectory planning and resource optimization.

However, whether using heuristic approaches or efficient approximation algorithms, such work still fundamentally fails to address the energy constraints of UAVs, as they merely treat battery capacity as a single constraint.

Obstacle-avoidance trajectory planning for energy-constrained UAVs: Some studies have designed obstacle avoidance algorithms for UAVs while accounting for energy constraints. Most existing obstacle avoidance algorithms in the field of UAV fall into two categories.

1) *Sampling-based algorithms:* The Artificial Potential Field (APF) method, commonly used in autonomous vehicle driving [22], has the drawback of a slow convergence rate in complex environments. And for the typical sampling-based algorithm RRT [23] and its variants [24], [25], [26], however, the path calculated by the RRT algorithm is often rugged. Moreover, due to the randomness of the sampling algorithms, their convergence speed and efficiency are also subject to certain limitations.

2) *Heuristic algorithms:* Currently, a series of Genetic Algorithm (GA) [27] variants have been applied in the UAV domain [28], [29], [30], [31]. However, due to the uncontrollability of genetic crossover and mutation, genetic algorithms often exhibit poor local optimization capability and slow convergence speed [27]. Inspired by the predatory behavior of birds, the Particle Swarm Optimization (PSO) algorithm [32] has been proposed. For example, Pan et al. [6] jointly optimized UAV power allocation and trajectories in wireless-powered communication networks considering obstacles using PSO. However, the classic PSO algorithm tends to converge prematurely and get trapped in local optima. On the other hand, the Simulated

Annealing (SA) algorithm, based on the annealing process of physical phenomena, can probabilistically escape local optima [33]. Consequently, some studies have combined the PSO algorithm with the SA algorithm [34]. Although heuristic algorithms can address more complex optimization problems, they still exhibit deficiencies compared with approximation algorithms that can find quality-guaranteed solutions within a reasonable time.

To sum up, while these studies have explored more complex mission scenarios for UAVs, they either treat energy merely as a constraint without addressing replenishment or rely on intelligent algorithms without establishing performance bounds. Consequently, the proposed solutions are not applicable to our work.

Deployment and planning of UAVs with wireless charging stations: Lahmeri et al. [10] proposed a laser-powered UAV deployment strategy to balance energy supply and wireless coverage. Song et al. [9] introduced a UAV deployment strategy to maximize k-coverage duration by considering charging and working as two distinct states. However, these studies solely focus on the field of wireless charging itself or merely consider charging as a static state, failing to consider how to plan UAV wireless charging trajectories during actual mission execution. Subsequently, these studies cannot be applied to the problem in our paper.

Charging UAVs during trajectory planning: Du et al. [11] proposed a trajectory optimization algorithm to maximize the number of IoT devices serviced before their deadlines by collecting solar energy. However, solar energy collection is far less efficient than current wireless charging technologies. Hu et al. [12] considered wireless power transfer (WPT) between access points (APs) and UAVs in mobile edge computing (MEC) networks, using laser and radio frequency (RF) technologies. However, their solution suffers from low charging efficiency when UAVs are far from APs, making it unsuitable for our problem.

III. SYSTEM MODEL

Suppose there are N devices on the ground denoted as a set $\mathcal{U} = \{u_1, u_2, \dots, u_N\}$, where the i th device is located at coordinate $u_i = (x_i, y_i, 0)$. We assume that each device transmits data to the UAV with equal data volume D and transmission power P_{trans} . All devices are served by a single UAV flying at a fixed height H starting from base station s_0 .

Given that the positions of all devices are known, the target area is partitioned into grids with grid cell length l with $l < 2r$, where r represents the UAV's data transmission range projected on the ground. Clusters are formed based on the proximity principle, grouping devices according to their distance from the central point. The data collection points for the UAV are designated as the grid centers containing devices within the clusters. We denote $\mathcal{SP} = \{s_1, s_2, \dots, s_M\}$ as the grid centers at a fixed height H with coordinate $s_j = (x_j, y_j, H)$ where the UAV collects data.

Due to the substantial workload of the UAV, there have been deployed K wireless charging stations denoted

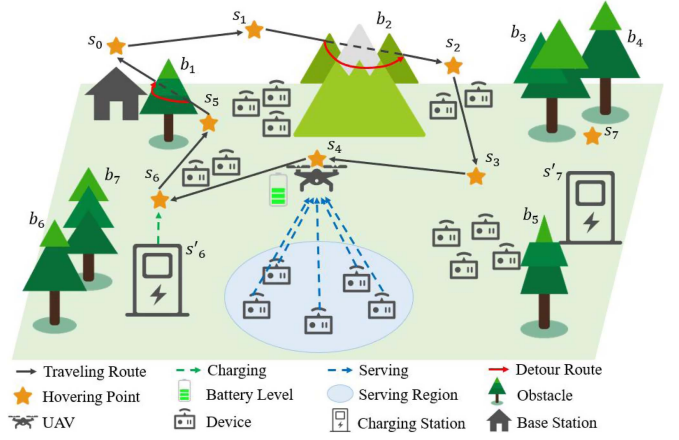


Fig. 1. Scenario.

as $\mathcal{CS} = \{s'_{M+1}, s'_{M+2}, \dots, s'_{M+K}\}$, with coordinates $s'_k = (x_k, y_k, H_{cs})$ with fixed height H_{cs} to replenish the UAV's energy. The charging positions of these stations are represented by the set $\mathcal{CP} = \{s_{M+1}, s_{M+2}, \dots, s_{M+K}\}$, with coordinate $s_k = (x_k, y_k, H)$. Clearly, the base station s_0 , serving positions $\mathcal{SP}$, and the wireless charging positions $\mathcal{CP}$ together constitute the entire hovering positions $\mathcal{HP}$ of the UAV.

Additionally, the real-world scenario contains obstacles such as tall buildings or trees. We use circumscribed cylinders to represent these obstacles, since cylindrical projections align with typical urban structures (e.g., towers, chimneys), as well as it is a good trade-off between realism and computational tractability in complex environments. These cylinders are denoted as $\mathcal{O} = \{b_1, b_2, \dots, b_Q\}$, with r_q representing the radius of the circumscribed cylinder of obstacle b_q . And we configure the cylindrical shell such that twice its radius (i.e., the diameter) always conservatively exceeds the maximum lateral dimension of the original obstacle, ensuring full encapsulation with a safety margin. Without ambiguity and for concise representation, we also use tuple $\langle b_q, r_q \rangle$ to represent the q th obstacle. Note that if some obstacles are too close, i.e., the circumscribed cylinders of these obstacles intersect or tangent, they are regarded as one obstacle in our model. Moreover, we only consider feasible hovering positions, i.e., positions not inside any circumscribed cylinders of the obstacles. Note that we will discuss the case of non-cylindrical approximate obstacles in Section VII.

The specific problem we address is to plan an obstacle-avoidance trajectory for the UAV that starts from the initial base station s_0 , collects data from all devices in the area—visiting all serving points s_1 through s_M —and then returns to the starting point. During this process, the UAV must maintain a sufficient battery level while balancing the total time spent and the number of selected charging stations.

We present a specific example with the scenario illustrated in Fig. 1. In the target area, there are $K = 2$ charging stations, $N = 16$ devices, $Q = 7$ obstacles as well as one base station and one UAV. We partition all devices into hovering positions at a constant height H based on the UAV's communication radius r (projected on the ground) and set one hovering point for each

charging station, resulting in $M = 8$ hovering locations, labeled s_0 through s_7 . The two charging stations are labeled s'_6 through s'_7 . In the figure, the trajectory passes through charging station s'_6 , while charging station s'_7 is not selected.

A. Wireless Data Collection Model

Due to the high flexibility of UAVs and their flight altitude, the likelihood of obstructions in ground-to-air communications is significantly reduced. We assume that the wireless communication channel between the UAV and ground devices in the service area is dominated by Line-of-Sight (LoS) links [1]. Therefore, the channel power gain from the UAV to each device is modeled as free-space trajectory loss. Given the UAV's data transmission position coordinate s_j and the coordinates of the devices within its transmission range u_i , the channel power gain $g_{i,j}$ from the UAV to each device is given by (1).

$$g_{i,j} = \frac{g_0}{(x_j - x_i)^2 + (y_j - y_i)^2 + H^2}, \quad x_i, y_i \in U_s(s_j), \quad (1)$$

where g_0 is the channel power gain at a reference distance of 1 m, and $U_s(s_j)$ denotes the set of coordinates of all devices u_i within the data transmission region of s_j . Given the transmission power P_{trans} of the devices, the signal-to-noise ratio (SNR) $\gamma_{i,j}$ for ground device i can be described as follows.

$$\gamma_{i,j} = \frac{P_{trans}g_{i,j}}{\sigma^2}, \quad (2)$$

where σ^2 is the noise power for each ground device. If the SNR of device i is not less than the threshold γ_{th} , then ground device i is considered to be covered by the UAV.

Therefore, we can determine the UAV's wireless coverage range r , i.e., the maximum ground range that meets the threshold SNR γ_{th} .

$$r = \sqrt{\frac{P_{trans}g_0}{\gamma_{th}\sigma^2} - H^2}, \quad (3)$$

Given the coordinates of data transmission position s_j and all ground devices, we can define the set of devices $U_s(s_j)$ within the service range of s_j as (4).

$$U_s(s_j) = \{u_i | (x_i - x_j)^2 + (y_i - y_j)^2 \leq r^2\}. \quad (4)$$

Meanwhile, we select OFDMA as the uplink spectrum access method, allocating independent subcarriers to each device. For each device within the region of s_j , the data transmission rate $c_{i,j}$ can be expressed as follows.

$$c_{i,j} = \frac{W}{|U_s(s_j)|} \log_2(1 + \gamma_{i,j}) \geq c. \quad (5)$$

where W is the bandwidth and c represents the minimum transmission rate of the devices.

B. Energy Consumption Model of UAV

The total energy consumption of a UAV can be divided into two parts. The first part is the energy consumed for communication, including signal modulation, demodulation, and radiation.

The second part is the energy required for the UAV to overcome gravity and provide thrust, ensuring that it can hover and fly. It is important to note that the energy consumption for communication is significantly less than the energy required to overcome gravity and provide thrust.

First, according to [35], the propulsion power consumption $P_{prop}(v)$ of a rotary-wing UAV at a uniform height as a function of speed v is given by (6).

$$P_{prop}(v) = P_{blade} \left(1 + \frac{3v^2}{v_{tip}^2} \right) + P_I \left(\sqrt{1 + \frac{v^4}{4v_0^4}} - \frac{v^2}{2v_0^2} \right)^{\frac{1}{2}} + \frac{1}{2} \zeta \rho R_{solid} R_{area} v^3, \quad (6)$$

where P_{blade} and P_I represent the blade profile power and the induced power in hover, respectively; v_{tip} , v_0 , ζ and ρ denote the tip speed of the rotor, the mean rotor induced velocity in hover, the fuselage drag ratio, and the air density, respectively; R_{solid} and R_{area} represent the rotor solidity and the area of the rotor disk, respectively.

In our model, the UAV operates in two states: hovering and flying. Based on the UAV propulsion power as a function of speed, the energy consumption for the UAV hovering at each data collection position in $\mathcal{SP}$ can be calculated by (7).

$$E_h^{sp} = P_{prop}(0) \cdot t_h^{sp} + P_{rx} \cdot t_h^{sp} = (P_{blade} + P_I + P_{rx}) \frac{D}{c}, \quad (7)$$

where P_{rx} is the data reception power of the UAV, and t_h^{sp} is the hovering time at each location in $\mathcal{SP}$, and we assume that it is determined by the device with the longest transmission time, i.e., transmitting at the slowest speed, which can be expressed as $t_h^{sp} = \frac{D}{c}$ (where D is the equal data volume of each ground device).

Since the calculation of charging time is related to the energy consumption during hovering while charging, and the amount of energy consumed during hovering is in turn determined by the charging time, these two variables are highly correlated. In order to make full use of the energy of the UAV, we have standardized the charging time for the drone at each charging station to the maximum charging time t_{max}^{harv} . And the energy consumption of UAV at each location in $\mathcal{CP}$ can be calculated by (8).

$$E_h^{cp} = P_{prop}(0) \cdot t_{max}^{harv} = (P_{blade} + P_I) \frac{B_{max}}{P_{harv}}. \quad (8)$$

Without loss of generality, we consider the UAV's change in speed during takeoff and landing. Assuming the UAV's acceleration (or deceleration) is a , the flight energy consumption $E_f^{i,j}$ between any two hovering positions s_i and s_j is shown in (9).

$$E_f^{i,j} = \int_0^{v/a} P_{prop}(at) dt + P_{prop}(v) \cdot t_f^{i,j} + \int_0^{v/a} P_{prop}(v - at) dt, \quad (9)$$

where $t_f^{i,j}$ is the constant speed flight time. We assume the constant flight speed of the UAV is v . Therefore, the constant

speed flight time between s_i and s_j can be expressed as $\frac{a \cdot d_{ij} - v^2}{av}$. Here, d_{ij} represents the distance between position s_i and s_j .

C. Wireless Charging Model

Currently, various wireless charging technologies exist, such as RF (radio frequency) charging and laser charging. Given the capability of laser to achieve long-distance and high-power transmission, we know that the charging station is at a fixed height H_{cs} , and is equipped with a laser transmitter that transmits laser energy to the UAV at a constant power P_0 , with power transmission occurring only when the UAV is directly above the charging station. The harvested laser charging power P_{harv} can be expressed as:

$$P_{harv} = \frac{\eta A_{rec} \vartheta P_0 e^{-\kappa_{tt}(H-H_{cs})}}{(A_{emit} + \beta(H-H_{cs}))^2}, \quad (10)$$

where η is the UAV's energy conversion efficiency, A_{rec} is the area of the laser receiver lens, ϑ is the combined transmission and receiver optical efficiency, A_{emit} is the initial laser beam size, κ_{tt} is the atmospheric attenuation coefficient, β is the beam divergence angle [10], [36], [37]. Thus, the total energy $\widehat{E}_u$ harvested by the UAV during the charging time t_{max}^{harv} can be calculated as (11).

$$\widehat{E}_u = P_{harv} t_{max}^{harv}. \quad (11)$$

IV. PROBLEM FORMULATION

First, we define a weighted directed complete graph G , where the vertices in the graph represent all potential locations in the UAV trajectory, and each edge represents the direct flight path between each pair of vertices, with the edge weight representing the flight distance. We define a binary variable $x_{ij} = 1$ to indicate that the edge between vertices s_i and s_j is part of the trajectory; $x_{ij} = 0$, otherwise. Let $\mathcal{S}$ be an already computed complete and ordered set of path vertices. The total distance of the trajectory with vertices in $\mathcal{S}$ can be expressed as follows.

$$\mathcal{L}(\mathcal{S}) = \sum_{i=0}^{M+K} \sum_{j=0, j \neq i}^{M+K} w(i, j) \cdot x_{ij}, \quad (12)$$

where $w(i, j)$ is the weight (i.e., total distance) of the edge between vertices s_i and s_j , which can be expressed as $w(i, j) = d_{ij}$. And the total time cost of the set $\mathcal{S}$ can be expressed as follows.

$$\begin{aligned} T(\mathcal{S}) = & \sum_{i=1}^M t_h^{sp} + \sum_{i=0}^{M+K} \sum_{j=M+1, j \neq i}^{M+K} x_{ij} \cdot t_{max}^{harv} \\ & + \sum_{i=0}^{M+K} \sum_{j=0, j \neq i}^{M+K} \left(\frac{v}{a} + \frac{w(i, j)}{v} \right) \cdot x_{ij}, \end{aligned} \quad (13)$$

where $(\frac{v}{a} + \frac{w(i, j)}{v})$ is the flight time of the UAV between vertices s_i and s_j .

Next, we define B_i as the remaining energy of the UAV when it leaves the hovering position s_i . Assume that the UAV flies to

s_j in the next step. B_j can be calculated as (14) shows.

$$B_j = \begin{cases} B_i - E_f^{ij} - E_h^{sp}, & 1 \leq j \leq M \text{ and } x_{ij} = 1, \\ B_{max}, & M+1 \leq j \leq M+K \text{ and } x_{ij} = 1. \end{cases} \quad (14)$$

We assume that the UAV starts with a full battery capacity and that the remaining energy of the UAV is non-negative when it leaves each data collection location. Specifically, $B_0 = B_{max}$ and $B_i > 0$ for $1 \leq i \leq M$.

Then, we require the UAV to move toward charging stations to replenish energy, so we set a reward for reaching each charging station as shown in (15).

$$Reward(\mathcal{S}) = \sum_{i=0}^{M+K} \sum_{j=M+1, j \neq i}^{M+K} x_{ij} \cdot \tau, \quad (15)$$

where τ is a constant representing the reward for reaching each charging station. Our objective is to optimize the UAV trajectory, that is, which charging stations to visit should be carefully planned, thus balancing the total travel time and the number of selected charging stations. We introduce a weight λ to jointly optimize these two variables and define the objective function f as (16) shows.

$$f(\mathcal{S}) = Reward(\mathcal{S}) - \lambda \cdot T(\mathcal{S}), \quad \lambda > 0. \quad (16)$$

We assume that each charging station can be visited at most once. The Practical Optimizing UAV Trajectory in Wireless Charging Networks (POTWCN) problem can be formalized as problem **P1**.

$$(\mathbf{P1}) \quad \max \quad f(\mathcal{S}) \quad (17)$$

$$\text{s.t.} \quad B_0 = B_{max}, \quad (18)$$

$$B_i > 0, 1 \leq i \leq M, \quad (19)$$

$$M+1 \leq |\mathcal{S}| \leq M+K+1, \quad (20)$$

$$\sum_{j=1, j \neq i}^{M+K} x_{ij} = \sum_{j=1, j \neq i}^{M+K} x_{ji} = 1, 0 \leq i \leq M, \quad (21)$$

$$\sum_{j=1, j \neq i}^{M+K} x_{ij} = \sum_{j=1, j \neq i}^{M+K} x_{ji}, 0 \leq i \leq M+K, \quad (22)$$

$$x_{ii} = 0, x_{ij} \in \{0, 1\}, \quad 0 \leq i, j \leq M+K, i \neq j. \quad (23)$$

In these constraints, (18) and (19) ensure the energy limitations, (20) limits the maximum number of charging stations that can be added, (21)-(22) enforce the trajectory selection constraints, and (22) specifies the binary nature of the decision variables.

Next, we have the following theorem to indicate the hardness of the problem POTWCN.

Theorem IV.1: The POTWCN problem is NP-hard.

Proof: To prove that the POTWCN problem is NP-hard, we reduce the Subset TSP problem [38], which is a known NP-hard problem, to the problem POTWCN. The Subset TSP problem is defined as follows: given a weighted graph, find the shortest tour that visits every vertex in a specified subset $\mathcal{S} \subseteq \mathcal{T}$ at

least once, where vertices not in $\mathcal{S}$ (i.e., $\mathcal{T} - \mathcal{S}$) may or may not be included in the tour to achieving our goal. To reduce Subset TSP to POTWCN, we first convert the input of the Subset TSP problem into the input of the POTWCN problem, which includes the graph G and the vertex subset $\mathcal{S}$, with edge weights representing negative benefits. In this conversion, the vertices in $\mathcal{S}$ are treated as device locations, and the remaining vertices $\mathcal{T} - \mathcal{S}$ are considered as optional charging stations. We examine a special case of the POTWCN problem where the UAV's initial energy is sufficient to cover the entire Subset TSP tour without requiring any recharging. In this case, the solution to the POTWCN problem can be directly translated into a solution for the Subset TSP problem. This reduction demonstrates that solving the POTWCN problem is at least as hard as solving the Subset TSP problem, thereby proving that the POTWCN problem is NP-hard. ■

V. SOLUTION

A. Algorithm Design

1) *Obstacle-Aware Weighted Graph Generation Algorithm (OWGGA)*: First, we propose an algorithm that constructs a weighted complete graph G with vertices set $\mathcal{HP}$, where the edge weights account for potential obstacles between vertices, as shown in Algorithm 1.

The algorithm iterates through all vertex pairs s_i and s_j in the hyperplane $\mathcal{HP}$ (line 1) and computes the actual flight distance between them. It first checks the case where no obstacles exist between the two points (line 3).

When an intersection between the line segment $s_i s_j$ and obstacles in set $\mathcal{O}$ is detected, the algorithm identifies the intersecting obstacles set $\mathcal{O}$ and calculates the intersection points $s_{ij,1}^p, \dots, s_{ij,Q_{ij}-1}^p$ on the cylindrically projected circles of the obstacles. These points are closer to s_j on each circle, excluding the circle nearest to s_j . The initial intersection point is set as the starting point of the segment (lines 5-7).

For each traversed obstacle b_{q_k} , the algorithm computes the tangent points $s_{ij,k,1}$ and $s_{ij,k,2}$ on obstacle b_{q_k} , corresponding to the previous intersection point $s_{ij,k-1}^p$ and the current intersection point $s_{ij,k}^p$, respectively (line 9). It then calculates the straight-line distances $L_{s_{ij,k-1}^p, s_{ij,k,1}}^p$, $L_{s_{ij,k,1}, s_{ij,k,2}}^p$, and the arc length $L_{s_{ij,k,1}, s_{ij,k,2}}$ between the tangent points (lines 10-12). Finally, the algorithm sums these three values to obtain the total detour distance, which is returned as the edge weight between vertices s_i and s_j in graph G , representing the final flight distance (line 13).

This computation considers obstacles along the path, ensuring that the edge weights accurately reflect the potential detour the UAV might need to take in the environment.

2) *Adaptive Charging Station Algorithm (ACSA)*: Then, we propose a greedy-based approximation algorithm that achieves a near-optimal solution for maximizing the objective function by limiting the number of charging stations along the route. We select one charging point with the maximum incrementation of the objective function $f(\mathcal{S})$ in each iteration. The pseudocode of our proposed algorithm is presented in Algorithm 2.

Algorithm 1: Obstacle-Aware Weighted Graph Generation Algorithm (OWGGA).

Input : Hovering positions
 $\mathcal{HP} = \{s_0, s_1, \dots, s_M, s_{M+1}, \dots, s_{M+K}\}$,
obstacles set $\mathcal{O} = \{b_1, \dots, b_Q\}$, and obstacle
circumscribed cylinder radius set
 $\mathcal{R} = \{r_1, \dots, r_Q\}$.

Output: Weighted complete graph
 $G = (\mathcal{HP}, \mathcal{HP} \times \mathcal{HP})$ with edge weight
matrix w .

```

1 foreach pair of vertices  $s_i$  and  $s_j$  in  $\mathcal{HP}$  do
2   if  $s_i s_j$  does not intersect any obstacle in  $\mathcal{O}$  then
3      $w(i, j) = w(j, i) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$ ;
4   else
5     Find the obstacles
6      $\mathcal{O}_{ij} = \{\langle b_{q_1}, r_{q_1} \rangle, \dots, \langle b_{q_{Q_{ij}}}, r_{q_{Q_{ij}}} \rangle\}$  that  $s_i s_j$ 
       intersects;
7     Compute intersection points of line  $s_i s_j$  and
       the obstacles in  $\mathcal{O}$ , and reserve the points
       nearer to  $s_j$  on each circle except the circle
       nearest to  $s_j$  as  $s_{ij,1}^p, \dots, s_{ij,Q_{ij}-1}^p$ ;
8     Set  $s_{ij,0}^p = s_i$  and  $s_{ij,Q_{ij}}^p = s_j$ ;
9     for  $k = 1$  to  $Q_{ij}$  do
10      Compute tangent points  $s_{ij,k,1}$  and  $s_{ij,k,2}$ 
        on the circle of  $(b_{q_k}, r_{q_k})$  for  $s_{ij,k-1}^p$  and
         $s_{ij,k}^p$ ;
11       $L_{s_{ij,k-1}^p, s_{ij,k,1}}^p =$ 
         $\sqrt{(x_{s_{ij,k-1}^p}^p - x_{s_{ij,k,1}}^p)^2 + (y_{s_{ij,k-1}^p}^p - y_{s_{ij,k,1}}^p)^2}$ ;
12       $L_{s_{ij,k,1}, s_{ij,k,2}}^p =$ 
         $\sqrt{(x_{s_{ij,k,1}}^p - x_{s_{ij,k,2}}^p)^2 + (y_{s_{ij,k,1}}^p - y_{s_{ij,k,2}}^p)^2}$ ;
13       $L_{s_{ij,k,1}, s_{ij,k,2}} = r_{q_k} \cdot \angle s_{ij,k-1}^p b_{q_k} s_{ij,k}^p$ ;
14       $w(i, j) = w(j, i) =$ 
         $\sum_{k=1}^{Q_{ij}} (L_{s_{ij,k-1}^p, s_{ij,k,1}}^p + L_{s_{ij,k,1}, s_{ij,k,2}}^p + L_{s_{ij,k,1}, s_{ij,k,2}})$ ;
15 return  $G$  and  $w$ ;
```

First, the algorithm ACSA initializes the number of selected charging positions k , the set $\mathcal{S}$ to record the indices of vertices currently on the trajectory and $\mathcal{CP}'$ to record the candidate indices of all charging positions (line 1). Then, ASCA iteratively increases the number of charging stations in the set $\mathcal{S}$ until it exceeds the maximum number K . Within the loop, we select the charging position s_{i^*} that provides the maximum increment of the objective function, add it to $\mathcal{S}$, and remove it from $\mathcal{CP}'$ (lines 7-9). Next, we check if the energy constraints are met under the current number of charging stations, ensuring the UAV does not run out of battery during its flight. If the constraints are satisfied, the algorithm exits the loop; otherwise, it increases the number of allowed charging positions k and proceeds to the next iteration. Finally, the algorithm returns the final trajectory.

Algorithm 2: Adaptive Charging Station Algorithm (ACSA).

Input : Base station s_0 , serving positions set

$\mathcal{SP} = \{s_1, \dots, s_M\}$, charging positions set

$\mathcal{CP} = \{s_{M+1}, \dots, s_{M+K}\}$.

Output: Path vertices pair set $\mathcal{P}$.

1 Initialize $k = 0$, $\mathcal{S} = \mathcal{SP} \cup \{s_0\}$, and $\mathcal{CP}' = \mathcal{CP}$;

2 Calculate the edge weight matrix w by Alg. 1.

3 **repeat**

4 **if** $k == 0$ **then**

5 Calculate the trajectory $\mathcal{P}$ on graph

$G = (\mathcal{S}, \mathcal{S} \times \mathcal{S})$ with weight w by

 Christofides algorithm [15];

6 **else**

7 $s_{i^*} = \arg \max_{s_i \in \mathcal{CP}'} \{f(\mathcal{S} \cup \{s_i\}) - f(\mathcal{S})\}$;

 // $f(\cdot)$ is calculated based on
 the trajectory $\mathcal{P}$ generated by
 Christofides algorithm [15];

8 $\mathcal{S} = \mathcal{S} \cup \{s_{i^*}\}$;

9 $\mathcal{CP}' = \mathcal{CP}' \setminus \{s_{i^*}\}$;

10 **if** $\forall (s_{j_1}, s_{j_2}) \in \mathcal{P} : B_{j_2} > 0$ **then**

11 **break**;

12 $k = k + 1$;

13 **until** $k > K$;

14 **return** $\mathcal{S}, \mathcal{P}$;

B. Theoretical Analysis

In this section, we will give the theoretical analysis of the proposed algorithm, including the approximation ratio of the path length and the objective, as well as the time complexity.

1) *Path Length Analysis:* In this subsection, we will derive the approximation ratio of the detour path length of the proposed algorithm ASCA (Algorithm 2). We first give the following lemma.

Lemma V.1: Given any $(s_i, s_j) \in \mathcal{P}$, where $\mathcal{P}$ is the path vertices pair set, the approximation ratio is given by:

$$1 \leq \frac{w(i, j)}{d_{ij}^*} \leq \frac{\pi}{2}. \quad (24)$$

where $w(i, j)$ the length of the detour path between s_i and s_j calculated in Algorithm 1, and d_{ij}^* is the minimum detour path length between s_i and s_j .

Proof: Let d_{ij} denote the length of the straight line segment $s_i s_j$. It is obvious that $w(i, j) \geq d_{ij}^* \geq d_{ij}$. $w(i, j) = d_{ij}^* = d_{ij}$ if and only if there is no obstacle between s_i and s_j . Thus, we have:

$$1 \leq \frac{w(i, j)}{d_{ij}^*} \leq \frac{w(i, j)}{d_{ij}}. \quad (25)$$

For the latter inequality in (24), we first consider that only one obstacle lies between s_i and s_j . As shown in Fig. 2, we use the top view to describe. Suppose that the obstacle $\langle b_q, r_q \rangle$ lies between s_i and s_j where $(s_i, s_j) \in \mathcal{P}$, that is, the distance between the point b_q and the line $s_i s_j$ is no larger than the radius r_q . Draw the tangent lines to circle b_q from s_i and s_j with the

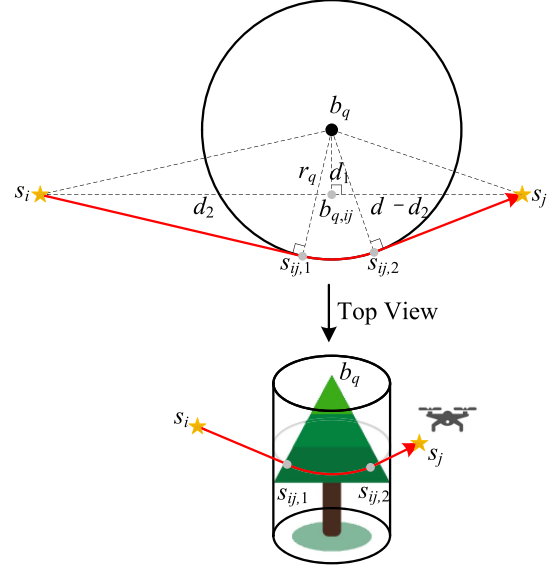


Fig. 2. Detour with one obstacle.

direction that $\angle s_i b_q s_j \leq \pi$, and we denote the tangent points as $s_{ij,1}$ and $s_{ij,2}$, respectively. Thus, the whole detour is $(s_i s_{ij,1} + s_{ij,1} s_{ij,2} + s_{ij,2} s_j)$. Draw the perpendicular line from b_q to $s_i s_j$ and we denote the perpendicular point as $b_{q,ij}$.

Here, we denote the euclidean distance d_{ij} of $s_i s_j$ as d to simplify. Let d_1 be the distance from b_q to the line $s_i s_j$, that is, the length of $b_q b_{q,ij}$ is d_1 . Let d_2 be the length of $s_i b_{q,ij}$, so the length of $s_j b_{q,ij}$ is $d - d_2$. Since s_i and s_j are available hovering positions, they must not be inside the obstacle circle. So, we have the constraints:

$$\begin{cases} 0 \leq d_1 \leq r_q, & (26) \\ d_2 \geq \sqrt{r_q^2 - d_1^2}, & (27) \\ d - d_2 \geq \sqrt{r_q^2 - d_1^2}. & (28) \end{cases}$$

We write the detour length $w(i, j)$ as a function of d_1 and d_2 , which is $L(d_1, d_2)$ as shown in (29).

$$\begin{aligned} L(d_1, d_2) = & \sqrt{d_1^2 + d_2^2 - r_q^2} + \sqrt{d_1^2 + (d - d_2)^2 - r_q^2} \\ & + r_q \left(\arccos \frac{d_1}{\sqrt{d_1^2 + d_2^2}} + \arccos \frac{d_1}{\sqrt{d_1^2 + (d - d_2)^2}} \right. \\ & \left. - \arccos \frac{r_q}{\sqrt{d_1^2 + d_2^2}} - \arccos \frac{r_q}{\sqrt{d_1^2 + (d - d_2)^2}} \right). \end{aligned} \quad (29)$$

In the following, we will prove the approximation ratio by two steps:

- 1) Fixing the length of $s_i s_j$, i.e., d , find the case, i.e., the value of d_1 and d_2 , that $L(d_1, d_2)$ is maximized. Suppose in this case, $d_1 = d_1^*$ and $d_2 = d_2^*$.
- 2) With the maximum value $L(d_1^*, d_2^*)$, find the maximum ratio of $L(d_1^*, d_2^*)/d \triangleq f_\gamma(d)$, that is, changing d to get

the maximum ratio. Thus, we obtain the maximum value of $w(i, j)/d_{ij}$.

For the first step, we first fix d_2 , then calculate the distance from b_q to line $s_i s_j$, i.e., the value of d_1 , that maximizes $L(d_1, d_2)$. Note that all the derivatives in the following are defined on the open interval for d_1 and d_2 to avoid zero values in the denominator.

$$\begin{aligned} \frac{\partial L(d_1, d_2)}{\partial d_1} &= \frac{d_1}{\sqrt{d_1^2 + d_2^2 - r_q^2}} + \frac{d_1}{\sqrt{d_1^2 + (d - d_2)^2 - r_q^2}} \\ &\quad - r_q \left(\frac{d_2}{d_1^2 + d_2^2} + \frac{d - d_2}{d_1^2 + (d - d_2)^2} \right) \\ &\quad + \frac{r_q d_1}{(d_1^2 + d_2^2) \sqrt{d_1^2 + d_2^2 - r_q^2}} \\ &\quad + \frac{r_q d_1}{(d_1^2 + (d - d_2)^2) \sqrt{d_1^2 + (d - d_2)^2 - r_q^2}} \Bigg) \\ &= \frac{d_1 \sqrt{d_1^2 + d_2^2 - r_q^2} - r_q d_2}{d_1^2 + d_2^2} \\ &\quad + \frac{d_1 \sqrt{d_1^2 + (d - d_2)^2 - r_q^2} - r_q (d - d_2)}{d_1^2 + (d - d_2)^2}. \end{aligned} \quad (30)$$

Let $\frac{\partial L}{\partial d_1} = 0$, we only get $d_1 = r_q$ when $d_1 \in [0, r_q]$. When $d_1 = 0$, $\frac{\partial L}{\partial d_1} = -r_q(1/d_2 + 1/(d - d_2)) < 0$, that is, $\frac{\partial L}{\partial d_1} \leq 0$ when $d_1 \in [0, r_q]$. Thus, $L(d_1, d_2)$ is a monotone decreasing function when d_2 is fixed and the maximum is obtained when $d_1 = 0$, i.e., the line $s_i s_j$ crosses the circle center.

Next, we calculate the value of d_2 that maximizes $L(0, d_2)$.

$$\begin{aligned} L(0, d_2) &= \sqrt{d_2^2 - r_q^2} + \sqrt{(d - d_2)^2 - r_q^2} \\ &\quad + r_q \left(\pi - \arccos \frac{r_q}{d_2} - \arccos \frac{r_q}{d - d_2} \right). \end{aligned} \quad (31)$$

Similarly, we also calculate the partial derivative.

$$\begin{aligned} \frac{\partial L(0, d_2)}{\partial d_2} &= \frac{d_2}{\sqrt{d_2^2 - r_q^2}} - \frac{d - d_2}{\sqrt{(d - d_2)^2 - r_q^2}} \\ &\quad - \frac{r_q/d_2^2}{\sqrt{1 - (r_q/d_2)^2}} + \frac{r_q/(d - d_2)^2}{\sqrt{1 - (r_q/(d - d_2))^2}} \\ &= \frac{\sqrt{d_2^2 - r_q^2}}{d_2} - \frac{\sqrt{(d - d_2)^2 - r_q^2}}{d - d_2}. \end{aligned} \quad (32)$$

Let $\frac{\partial L(0, d_2)}{\partial d_2} = 0$, we get $d_2 = d/2$. To further check the monotonicity of the function $f(0, d_2)$, we calculate the second partial derivative.

$$\frac{\partial^2 L(0, d_2)}{\partial d_2^2} = \frac{\frac{d_2^2}{\sqrt{d_2^2 - r_q^2}} - \sqrt{d_2^2 - r_q^2}}{d_2^2}$$

$$\begin{aligned} &= -\frac{\frac{(d-d_2)^2}{\sqrt{(d-d_2)^2 - r_q^2}} + \sqrt{(d-d_2)^2 - r_q^2}}{(d-d_2)^2} \\ &= \frac{\frac{r_q^2}{d_2^2 \sqrt{d_2^2 - r_q^2}} + \frac{r_q^2}{(d-d_2)^2 \sqrt{(d-d_2)^2 - r_q^2}}}{(d-d_2)^2} > 0. \end{aligned} \quad (33)$$

Thus, $L(0, d_2)$ is a convex function and has the minimum value when $d_2 = d/2$ and $d_2 \in (r_q, d - r_q)$. The maximum value is:

$$\begin{aligned} L(0, r_q) &= L(0, d - r_q) \\ &= \sqrt{(d - r_q)^2 - r_q^2} + r_q \left(\pi - \arccos \frac{r_q}{d - r_q} \right). \end{aligned} \quad (34)$$

Subsequently, we have:

$$\begin{aligned} \frac{w(i, j)}{d_{ij}} &= \frac{L(d_1, d_2)}{d} \leq \frac{L(0, d_2)}{d} \leq \frac{L(0, r_q)}{d} \\ &= \frac{\sqrt{(d - r_q)^2 - r_q^2} + r_q \left(\pi - \arccos \frac{r_q}{d - r_q} \right)}{d}. \end{aligned} \quad (35)$$

For the second step, we denote $f_\gamma(d) \triangleq \frac{L(0, r_q)}{d}$ in (35) to explore the maximum value of this function. Due to the constraints of (27) and (28), we have $d \geq 2r_q$ when $d_1 = 0$.

$$\begin{aligned} f'_\gamma(d) &= \frac{1}{d^2} \\ &\quad \left(d \left(\frac{d - r_q}{\sqrt{(d - r_q)^2 - r_q^2}} - \frac{r_q}{(d - r_q)^2 \sqrt{1 - (r_q/(d - r_q))^2}} \right) \right. \\ &\quad \left. - \sqrt{(d - r_q)^2 - r_q^2} - r_q \left(\pi - \arccos \frac{r_q}{d - r_q} \right) \right) \\ &= \frac{r_q}{d^2(d - r_q)} \left(\sqrt{(d - r_q)^2 - r_q^2} - (d - r_q) \right. \\ &\quad \left. \left(\pi - \arccos \frac{r_q}{d - r_q} \right) \right) \\ &< -\frac{r_q(d - r_q) \left(\pi - \arccos \frac{r_q}{d - r_q} - 1 \right)}{d^2(d - r_q)} < 0. \end{aligned} \quad (36)$$

The first inequality is because $\sqrt{(d - r_q)^2 - r_q^2} < d - r_q$. The last inequality is because $0 < r_q/(d - r_q) \leq 1$, so $1 < \pi/2 < (\pi - \arccos \frac{r_q}{d - r_q}) \leq \pi$, that is, $(\pi - \arccos \frac{r_q}{d - r_q} - 1) > 0$. Thus, $f_\gamma(d)$ is a monotone decreasing function when $d \geq 2r_q$ and the maximum value is $\pi/2$ when $d = 2r_q$. Therefore, combined with (25) and (35), we have:

$$1 \leq \frac{w(i, j)}{d_{ij}^*} \leq \frac{w(i, j)}{d_{ij}} \leq f_\gamma(d) \leq \frac{\pi}{2}. \quad (37)$$

The conclusion is proved when there is only one obstacle lying between s_i and s_j .

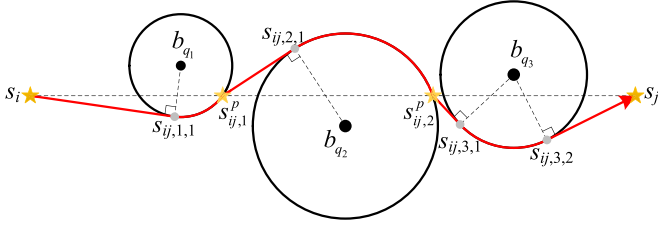


Fig. 3. Detour with several obstacles.

If there is more than one obstacle lying between s_i and s_j , as shown in Fig. 3, we separate $s_i s_j$ into several segments with $s_i s_{ij,1}^p, \dots, s_i s_{ij,Q_{ij}-1}^p$ where Q_{ij} is the number of obstacles lying between, and $s_{ij,1}^p, \dots, s_{ij,Q_{ij}-1}^p$ are points on the circles as well as on the line $s_i s_j$. On each segment, we calculate the detour length as previously shown. Thus, we have:

$$1 \leq \frac{w(i, j)}{d_{ij}^*} \leq \frac{w(i, j)}{d_{ij}} = \frac{\sum_{k=1}^{Q_{ij}-1} w(s_i, s_{ij,k}^p)}{\sum_{k=1}^{Q_{ij}-1} d_{s_i, s_{ij,k}^p}} \leq \frac{\pi}{2}. \quad (38)$$

This completes the proof. $\blacksquare$

Then, we will derive the total path length approximation ratio of Algorithm 2. We give the following theorem.

Theorem V.1: Algorithm 2 achieves $\frac{3\pi}{4}$ -approximation ratio of the total path length.

Proof: The approximation ratio of the total path length in the ACSA algorithm is derived from two main components: the approximation ratio for each edge's detour path length and the approximation ratio of Christofides' algorithm for solving the TSP.

Suppose the optimal solution and the solution obtained by Algorithm 2 are denoted as $\mathcal{S}^*$ and $\mathcal{S}_{SOL}$, respectively. The corresponding path vertices pair sets are denoted as $\mathcal{P}^*$ and $\mathcal{P}_{SOL}$, respectively. Moreover, the minimum detour length is denoted as function $\mathcal{L}^*(\cdot)$.

For the first part, based on Lemma V.1, the detour path length $w(i, j)$ generated by the Algorithm 2 satisfies:

$$1 \leq \frac{\mathcal{L}(\mathcal{S}^*)}{\mathcal{L}^*(\mathcal{S}^*)} = \frac{\sum_{(s_i, s_j) \in \mathcal{P}^*} w(i, j)}{\sum_{(s_i, s_j) \in \mathcal{P}^*} d_{ij}^*} \leq \frac{\pi}{2}. \quad (39)$$

For the second part, we consider the approximation ratio of the TSP. Christofides' algorithm provides a solution for the TSP with an approximation ratio of 1.5, that is,

$$1 \leq \frac{\mathcal{L}(\mathcal{S}_{SOL})}{\mathcal{L}(\mathcal{S}^*)} \leq \frac{3}{2}. \quad (40)$$

Combine with (39) and (40), we can derive the total path length approximation ratio:

$$1 \leq \frac{\mathcal{L}(\mathcal{S}_{SOL})}{\mathcal{L}^*(\mathcal{S}^*)} \leq \frac{3}{2} \times \frac{\pi}{2} = \frac{3\pi}{4}. \quad (41)$$

This completes the proof. $\blacksquare$

2) *Objective Approximation Ratio Analysis:* In this part, we prove the approximation ratio of the objective function. To begin with, we give some definitions and lemmas to support the submodularity of the objective function in P1.

Definition V.1: (Monotone submodular function [39]) A function $F: 2^U \rightarrow R_{\geq 0}$ is monotone submodular if for all $\mathcal{S} \subseteq \mathcal{T} \subseteq U$, $F(s_j|\mathcal{S}) \geq F(s_j|\mathcal{T})$, where $F(s_j|\mathcal{S}) = F(\mathcal{S} \cup \{s_j\}) - F(\mathcal{S})$ is the marginal improvement of adding element s_j to the set $\mathcal{S}$.

However, in most practical routing problems such as node coverage or TSP on graphs, the cost function is not submodular. The main limitation of existing work lies precisely in assuming the routing cost function to be submodular. On the other hand, there exist special cases of TSP that exhibit submodularity, such as when the graph forms a tree structure [39]. Inspired by this observation, we employ a natural relaxation of submodularity called α -submodularity.

Definition V.2: (α -submodularity [39]) For a function F , we define it as α -submodular for

$$\alpha = \min_x \min_{\mathcal{S}, \mathcal{T}: \mathcal{S} \subseteq \mathcal{T}} \frac{F(x|\mathcal{S})}{F(x|\mathcal{T})}. \quad (42)$$

Definition V.3: (α monotone submodular function properties [39]) For any α monotone submodular function F , the following statements hold.

- i) $F(s_j|\mathcal{S}) \geq \alpha F(s_j|\mathcal{T})$, $\forall \mathcal{S} \subseteq \mathcal{T} \subseteq U$ and $s_j \in U \setminus \mathcal{T}$.
- ii) $F(\mathcal{T}) \leq F(\mathcal{S}) + \frac{1}{\alpha} \sum_{s_j \in \mathcal{T} \setminus \mathcal{S}} F(s_j|\mathcal{S}) - \alpha \sum_{s_j \in \mathcal{S} \setminus \mathcal{T}} F(\mathcal{S} \cup \mathcal{T} \setminus s_j)$, $\forall \mathcal{S}, \mathcal{T} \subseteq U$.
- iii) $F(\mathcal{T}) \leq F(\mathcal{S}) + \frac{1}{\alpha} \sum_{s_j \in \mathcal{T} \setminus \mathcal{S}} F(s_j|\mathcal{S})$, $\forall \mathcal{S} \subseteq U, \mathcal{T} \subseteq U$.
- iv) $F(\mathcal{T}) \leq F(\mathcal{S}) - \alpha \sum_{s_j \in \mathcal{S} \setminus \mathcal{T}} F(s_j|\mathcal{S} \setminus s_j)$, $\forall \mathcal{T} \subseteq \mathcal{S} \subseteq U$.

Then, we prove the time cost function is an α monotone submodular function in the following lemma.

Lemma V.2: Given sufficient charging duration $t_{\max}^{harv}$, the time cost function $T(\cdot)$ defined in (13) is an α monotone submodular function.

Proof: Given two hovering positions sets $\mathcal{S}$ and $\mathcal{T}$ that $\mathcal{S} \subseteq \mathcal{T}$, when adding a charging station s_j , we obtain the marginal gain of time cost as follows:

$$T(s_j|\mathcal{S}) = t_{\max}^{harv} + \Delta t_f(\mathcal{S}), \quad (43)$$

where Δt_f is the flight time increment when optimizing the path. Then due to $T(s_j|\mathcal{S}) \geq 0$, we have

$$T(\mathcal{S}) \leq T(\mathcal{S} \cup \{s_j\}) \leq T(\mathcal{T}). \quad (44)$$

Since a larger number of elements in the set increases the likelihood of obtaining more optimized paths, thereby reducing the time increment, there exists $\alpha \in [0, 1]$ for any $\mathcal{S} \subseteq \mathcal{T}$ such that

$$\Delta t_f(\mathcal{S}) \geq \alpha \Delta t_f(\mathcal{T}). \quad (45)$$

So, we can obtain that

$$\begin{aligned} T(s_j|\mathcal{S}) - \alpha T(s_j|\mathcal{T}) &= (1 - \alpha)t_{\max}^{harv} + \Delta t_f(\mathcal{S}) - \alpha \Delta t_f(\mathcal{T}) \\ &\geq (1 - \alpha)t_{\max}^{harv} \geq 0. \end{aligned} \quad (46)$$

According to Definition V.3(i), the time cost function $T(\cdot)$ in (13) is an α monotone submodular function. $\blacksquare$

Next, we can obtain that Next, we attempt to prove the α -submodularity of the objective function. we have the following lemma.

Lemma V.3: For $\mathcal{S} \subseteq \mathcal{T}$, it holds that

$$f(\mathcal{S}) = \text{Reward}(\mathcal{S}) - \lambda \cdot T(\mathcal{S}) \leq f(\mathcal{T}), \quad (47)$$

where $\lambda \leq \frac{\text{Reward}(\mathcal{T}) - \text{Reward}(\mathcal{S})}{T(\mathcal{T}) - T(\mathcal{S})}$.

Proof: Since $\lambda \leq \frac{\text{Reward}(\mathcal{T}) - \text{Reward}(\mathcal{S})}{T(\mathcal{T}) - T(\mathcal{S})}$, and given that the time cost function is an α monotone submodular function (Lemma V.2), multiplying both sides of the inequality by the denominator (which preserves the inequality direction) yields, we have

$$\begin{aligned} \text{Reward}(\mathcal{T}) - \text{Reward}(\mathcal{S}) &\geq \lambda \cdot (T(\mathcal{T}) - T(\mathcal{S})) \\ &= \lambda \cdot T(\mathcal{T}) - \lambda \cdot T(\mathcal{S}). \end{aligned} \quad (48)$$

Then we can obtain that

$$\begin{aligned} f(\mathcal{S}) &= \text{Reward}(\mathcal{S}) - \lambda \cdot T(\mathcal{S}) \\ &\leq \text{Reward}(\mathcal{T}) - \lambda \cdot T(\mathcal{T}) = f(\mathcal{T}). \end{aligned} \quad (49)$$

Next, we prove the α -submodularity property of the objective function.

Lemma V.4: For $\mathcal{S} \subseteq \mathcal{T}$, it holds that

$$f(s_j|\mathcal{S}) \geq \alpha f(s_j|\mathcal{T}), \quad (50)$$

where $\lambda \leq \frac{\text{Reward}(s_j|\mathcal{S}) - \alpha \text{Reward}(s_j|\mathcal{T})}{T(s_j|\mathcal{S}) - \alpha T(s_j|\mathcal{T})}$, and α is from Definition V.2.

Proof: Due to $\lambda \leq \frac{\text{Reward}(s_j|\mathcal{S}) - \alpha \text{Reward}(s_j|\mathcal{T})}{T(s_j|\mathcal{S}) - \alpha T(s_j|\mathcal{T})}$, and similar to Lemma V.3, we obtain

$$\begin{aligned} \text{Reward}(s_j|\mathcal{S}) - \alpha \text{Reward}(s_j|\mathcal{T}) &\geq \lambda(T(s_j|\mathcal{S}) - \alpha T(s_j|\mathcal{T})) \\ &= \lambda T(s_j|\mathcal{S}) - \lambda \alpha T(s_j|\mathcal{T}). \end{aligned} \quad (51)$$

Then we can obtain that

$$\begin{aligned} f(s_j|\mathcal{S}) &= \text{Reward}(s_j|\mathcal{S}) - \lambda \cdot T(s_j|\mathcal{S}) \\ &\geq \alpha(\text{Reward}(s_j|\mathcal{T}) - \lambda \cdot T(s_j|\mathcal{T})) = \alpha f(s_j|\mathcal{T}). \end{aligned} \quad (52)$$

Therefore, according to Lemma V.3 and Lemma V.4, we find that by setting an appropriate value for λ , we can ensure that our objective function is a monotone increasing α -submodular function. By leveraging the diminishing marginal gains property of a monotone submodular function, we can naturally propose an approximation algorithm with guaranteed approximation bounds.

Theorem V.2: Algorithm 2 achieves $(1 - \frac{1}{e})$ -approximation ratio.

Proof: Suppose $\mathcal{S}^*$ is the optimal solution. $\mathcal{S}_i$ is the vertices set obtained after the i th iteration of Algorithm 2 and k is the number of selected charging stations in Algorithm 2. $f(\cdot)$ is the function defined in (16). According to Lemma V.3, Lemma V.4

and Definition V.3, we have

$$\begin{aligned} f(\mathcal{S}^*) &\leq f(\mathcal{S}_{i-1}) + \frac{1}{\alpha} \sum_{q \in \mathcal{S}^* / \mathcal{S}_{i-1}} (f(\mathcal{S}_{i-1} \cup \{q\}) - f(\mathcal{S}_{i-1})) \\ &\leq f(\mathcal{S}_{i-1}) + \frac{1}{\alpha} \sum_{q \in \mathcal{S}^* / \mathcal{S}_{i-1}} (f(\mathcal{S}_i) - f(\mathcal{S}_{i-1})) \\ &\leq f(\mathcal{S}_{i-1}) + \frac{k}{\alpha} (f(\mathcal{S}_i) - f(\mathcal{S}_{i-1})), \end{aligned} \quad (53)$$

where the first inequality holds due to Definition V.3, the second inequality holds due to the greedy rule, and the third inequality holds due to at most k charging stations to be selected in Algorithm 2. Now we subtract $k f(\mathcal{S}^*)$ from both sides of the equation simultaneously, notice that

$$\begin{aligned} f(\mathcal{S}_{i-1}) + \frac{k}{\alpha} f(\mathcal{S}_i) - \frac{k}{\alpha} f(\mathcal{S}_{i-1}) - \frac{k}{\alpha} f(\mathcal{S}^*) \\ \geq f(\mathcal{S}^*) - \frac{k}{\alpha} f(\mathcal{S}^*). \end{aligned} \quad (54)$$

Then according to mathematical induction, we can obtain that

$$\begin{aligned} f(\mathcal{S}_i) - f(\mathcal{S}^*) &\geq \left(1 - \frac{\alpha}{k}\right) (f(\mathcal{S}_{i-1}) - f(\mathcal{S}^*)) \\ &\geq \left(1 - \frac{\alpha}{k}\right)^i (f(\mathcal{S}_0) - f(\mathcal{S}^*)). \end{aligned} \quad (55)$$

Thus, we have that

$$f(\mathcal{S}_i) \geq \left(1 - \left(1 - \frac{\alpha}{k}\right)^i\right) f(\mathcal{S}^*). \quad (56)$$

When i approaches k , we derive the total approximation ratio of the algorithm, that is

$$\begin{aligned} f(\mathcal{S}_k) &\geq \left(1 - \left(1 - \frac{\alpha}{k}\right)^k\right) f(\mathcal{S}^*) \\ &\geq (1 - e^{-\alpha}) f(\mathcal{S}^*). \end{aligned} \quad (57)$$

In practical α approaches 1. The approximation ratio of Algorithm 2 reaches $1 - \frac{1}{e}$.

3) Time Complexity Analysis: In this subsection, we proceed to calculate the time complexity of the proposed algorithm. We propose the following theorem.

Theorem V.3: Given a UAV candidate positions graph $G = (\mathcal{HP}, \mathcal{HP} \times \mathcal{HP})$, Algorithm 2 achieves a time complexity of $O(Qn^2 + K^2n^3)$, where K is the number of charging stations and $n = |\mathcal{HP}|$.

Proof: Algorithm 2 can be separated into two parts: the initialization part (lines 1-2) and the main body (lines 3-13).

For the initialization part, we need to calculate the time complexity of Algorithm 1 (line 2). In Algorithm 1, we need to enumerate all pairs of vertices in $\mathcal{HP}$, so the total times of iterations for lines 1-13 is $C_n^2 = n(n-1)/2 = O(n^2)$. We then consider the worst case that the line between each pair of vertices intersects all the obstacles. Thus, the inner loop (lines 8-13) iterates Q times. The final time complexity of Algorithm 1 becomes $O(Qn^2)$.

For the main body, we still divide into two conditions: $k = 0$ and $k > 0$. When $k = 0$, line 5 is conducted with $O(n^3)$

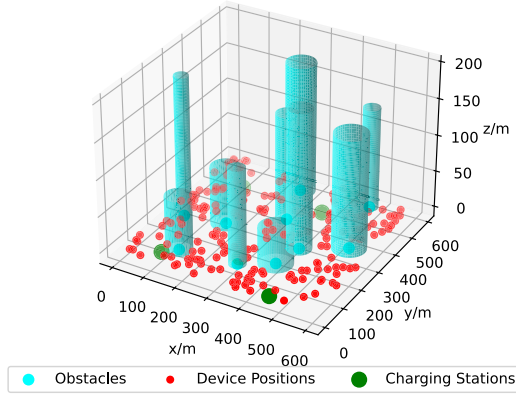


Fig. 4. Model Example.

(Christofides' algorithm), and line 10 is conducted M times with M edges in graph G . When $k > 0$, $f(\mathcal{S} \cup \{s_i\})$ is calculated $(K - (k - 1))$ times with $|\mathcal{CP}'| = (K - (k - 1))$ and Christofides' algorithm is conducted once in each calculation. Thus, the time complexity of line 7 is $O((K - (k - 1))n^3)$. In this case, line 10 is conducted $(M + k)$ times with $(M + k)$ edges in graph G . Therefore, the time complexity of the main body becomes $O((n^3 + M) + \sum_{k=1}^K ((K - (k - 1))n^3 + (M + k))) = O(K^2n^3)$.

To sum up, the time complexity of Algorithm 2 is $O(Qn^2 + K^2n^3)$. ■

VI. SIMULATION

In this section, we conduct simulations to evaluate our algorithm.

A. Experimental Settings

As shown in Fig. 4, we consider a ground device network that consists of default 200 devices, 10 obstacles, and 6 charging stations randomly deployed in a 600 m $\times$ 600 m square area. Table I lists the default settings of the parameters in this paper. We assume that the UAV is deployed at a depot s_0 initially. The UAV has an energy capacity of $B_{\max} = 200$ kJ, a constant flight speed of $v = 5$ m/s and a flying altitude of $H = 20$ m, with an acceleration of 0.5 m/s² during takeoff and a deceleration of 0.5 m/s² during landing. Additionally, through experimental validation, we demonstrated that the value of α in our objective function can exceed 0.9 and approach 1 when weight λ is 10.

B. Baselines

To evaluate the performance of the proposed algorithm, we introduce five comparison algorithms.

- 1) **CEDAN [40]**: The main idea of this algorithm is to first calculate the trajectory using the TSP algorithm. Then, using the function RADJUSTMENT, the edge weights of each trajectory are recalculated based on our predefined objective function values. Finally, a greedy selection is employed to readjust the sequence of node visits within the trajectory.

TABLE I
PARAMETER SETTINGS

Parameters	Values
Data collection area side length	600 m
Data transmission range of UAV r	70 m
The fixed altitude of UAV H	20 m
Acceleration of UAV a	0.5
Objective function weight λ	10
The blade profile power in hovering status P_{blade}	79.86
Induced power in hovering status P_I	88.63
Tip speed of the rotor blade v_{tip}	120 m/s
Mean rotor induced velocity in hover v_0	4.03
Fuselage drag ratio ζ	0.6
Air density ρ	1.225 kg/m ³
Rotor solidity R_{solid}	0.05
Rotor disc area R_{area}	0.503 m ²
Node data volume D	100 MB
Minimum transmission rate c	150 Mbps
Data reception power of the UAV P_{rx}	0.5 W
Energy conversion efficiencies η	0.8
Area of laser receiver's telescope or collection lens A_{rec}	10 ⁻² m ²
Combined transmission receiver optical efficiency ϑ	0.2
Energy transmit power of the CSs P_0	60 dBm
Size of the initial laser beam A_{emit}	0.05 m
Attenuation coefficient of the channel medium κ_{tt}	10 ⁻⁶ m ⁻¹
Angular spread β	3.4 $\times$ 10 ⁻⁵

- 2) **GA [27]**: The classical heuristic algorithm, Genetic Algorithm (GA), searches for optimal solutions through processes such as chromosome encoding, mutation, crossover, selection, and decoding.
- 3) **SA [33]**: Another heuristic algorithm, Simulated Annealing (SA), is based on the annealing process of physical phenomena.
- 4) **RRT [22]**: A sampling-based algorithm, Rapidly-exploring Random Tree (RRT) algorithm, computes detour paths in the presence of obstacles.
- 5) **PF [23]**: Another sampling-based algorithm, Artificial Potential Field (PF), has been commonly used in vehicle autonomous driving in recent years.

The result of algorithms is obtained based on a machine with 2.10 GHz Intel i7 single-core CPU and 32 GB RAM. Each value in the figures is the mean of the results out of 50 random network topologies of the same size.

C. Visualization of Final UAV Trajectories

First, we present the trajectory computed by our proposed algorithm under the settings of $K = 6$, $l = 70$ m, $B_{\max} = 200$ kJ, as shown in Fig. 5. In this scenario, our ACSA algorithm reduces the UAV flight distance by 56.5%, 66.2%, 70.4%, 4.77%, and 9.86% compared with CEDAN, SA, GA, RRT, and PF, respectively, while also reducing the task completion time by 16.7%, 50.1%, 51.4%, 6.25%, and 13.5%, respectively.

In the figure, the gray dots represent devices deployed in the scenario, the black-numbered points indicate UAV hovering positions, the blue-numbered points denote UAV wireless charging stations deployed on the ground, and the blue circular regions represent obstacles in the scenario.

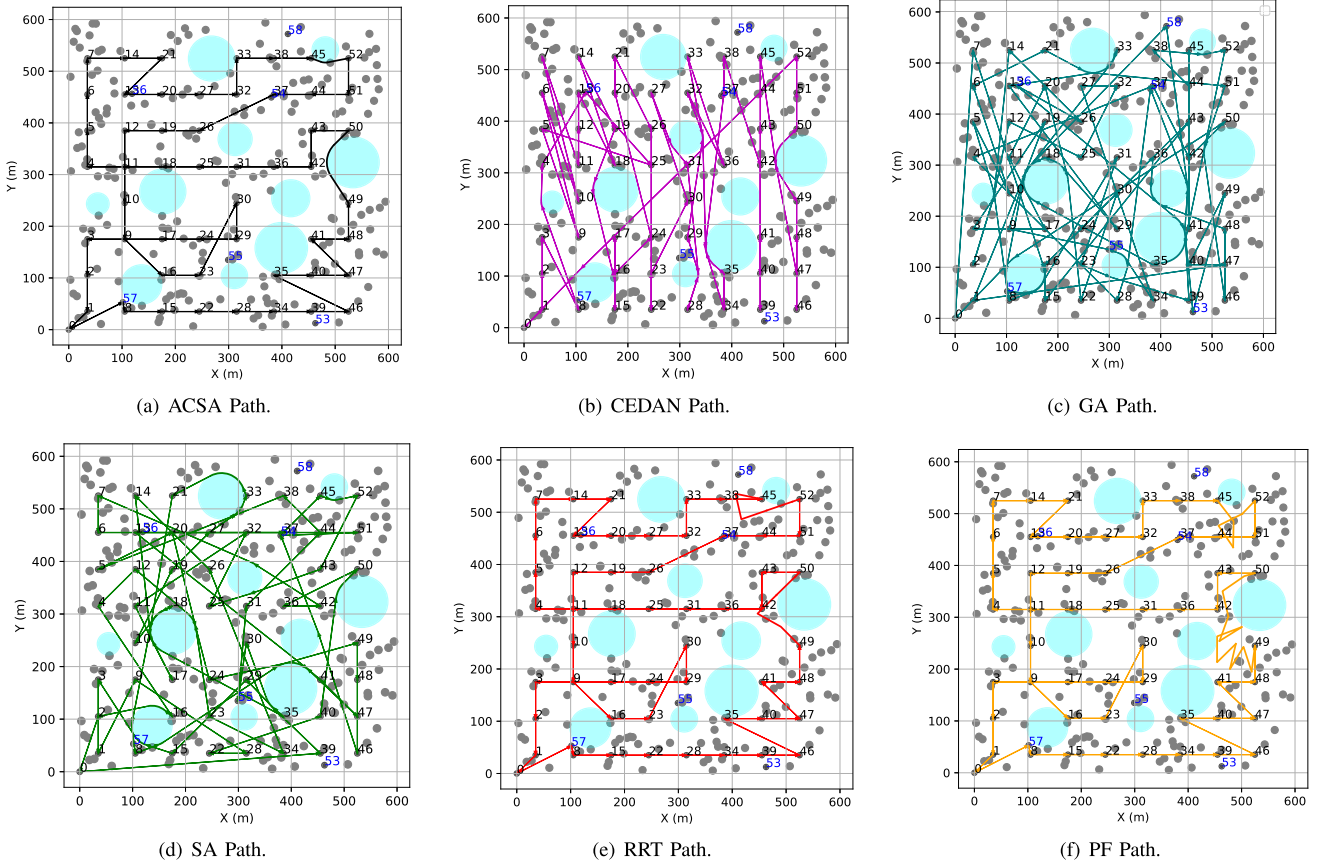


Fig. 5. Visualization of final UAV trajectories.

The UAV's starting base station is numbered as 0, and the figure includes 52 hovering positions numbered from 1 to 52 and charging stations numbered from 53 to 58.

From the Fig. 5, it can be seen that all six algorithms successfully traversed all hovering positions and generated a complete closed trajectory, ensuring each hovering position is visited only once. Moreover, all the trajectories effectively achieved obstacle avoidance. In Fig. 5(a), we present the trajectory computed by our proposed ACSA algorithm. Compared with the results of more random heuristic algorithms in Fig. 5(e) and (f), ACSA generates shorter and more structured detour paths. Moreover, as shown in Fig. 5(b), the trajectory of the CEDAN algorithm tends to fall into local optima, resulting in a longer path from 52 to 0. In Fig. 5(c) and (d), the heuristic algorithms GA and SA are displayed. These two algorithms generate several unnecessary detour trajectories under the constraint of a limited number of iterations due to the high complexity of heuristics. The results demonstrate that our proposed algorithm effectively solves the POTWCN problem and delivers superior performance.

D. Impact of Number of Charging Stations K

Our ACSA algorithm reduces the flight distance of the UAV by 42.57%, 49.60%, 54.36%, 14.86%, and 22.27% compared to CEDAN, SA, GA, RRT and PF, respectively, while also reducing

the task completion time by 36.45%, 43.68%, 49.64%, 15.53%, and 25.68%, respectively.

As shown in Fig. 6, we test the stability of the proposed algorithm by varying the number of charging stations K deployed in the scenario. The y -axis in the figure represents the completion time T , the minimum number of required charging stations k , and the total path length $\mathcal{L}$, respectively. In this scenario, we varied the total number of charging stations from 3 to 8, under the settings of $l = 70$ m, $B_{\max} = 150$ kJ, and assumed an average charging time of approximately 1.49 hours.

We observe that due to the strong correlation between time consumption and the number of charging stations, the overall trends of the curves in Fig. 6(a) and (b) are similar. Since the number of charging stations required remains relatively constant, increasing the total number of charging stations primarily increases the available options for each station addition. As such, the increase in charging station numbers from 3 to 8 in the same scenario has little impact on the results, aligning with our experimental observations.

However, CEDAN adjusts the path sequence after computing the trajectory, which may occasionally transform a reasonable path into an unreasonable one. Moreover, once CEDAN calculates the path and selects the charging station closest to the path, it does not dynamically optimize the path based on newly added charging stations in the scenario, causing the subsequent results to remain unchanged. Additionally, the remaining algorithms

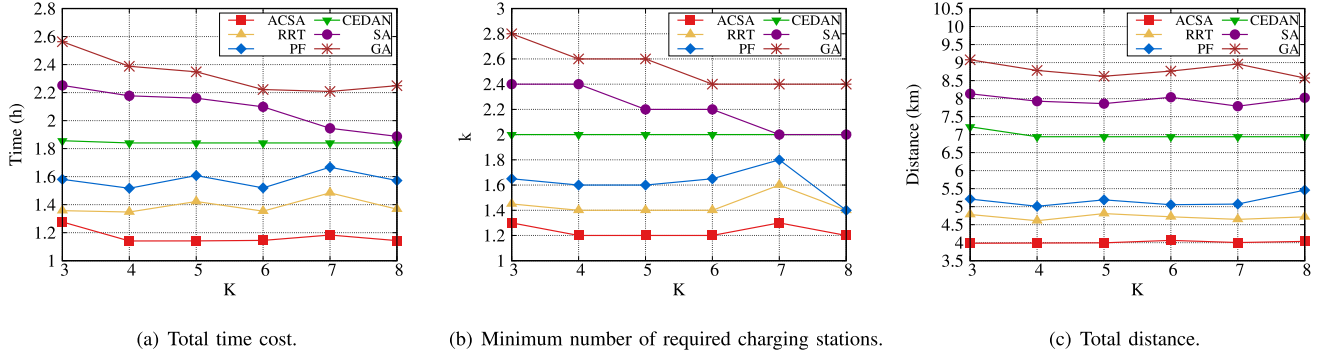


Fig. 6. Performance of different algorithms by varying the value of K from 3 to 8.

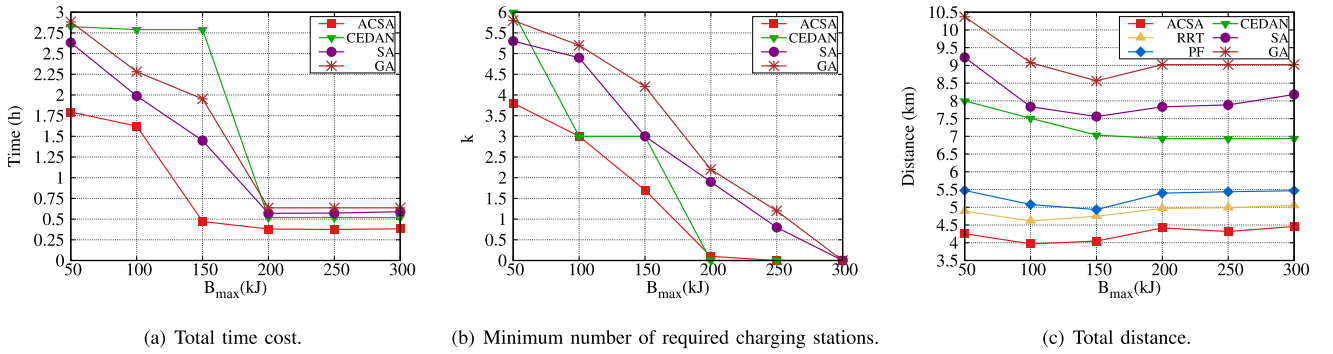


Fig. 7. Performance of different algorithms by varying the value of B_{max} from 100 kJ to 350 kJ.

demonstrate instability and produce suboptimal solutions. In contrast, our proposed algorithm achieves better stable results and converges to significantly better solutions.

E. Impact of Number of UAV Battery Capacity B_{max}

Our ACSA algorithm reduces the UAV flight distance by 41.09%, 47.33%, 53.79%, 13.00%, and 19.88% compared with CEDAN, SA, GA, RRT, and PF respectively, while also reducing the task completion time by 40.45%, 36.76%, 43.93%, 21.52%, and 26.15%, respectively.

As shown in Fig. 7(a), (b), and (c), we compare the total task completion time T , the minimum number of charging stations k required to ensure continuous operation, and total flight distance $\mathcal{L}$ under different UAV battery capacities. Since the time spent detouring around obstacles contributes a small proportion to the total task time, we separately compare the detour algorithms in Fig. 8 for better visualization. The results indicate that the trajectories computed by our algorithm require less detour time. From Fig. 7(a) and (b), it is evident that as the UAV battery capacity increases, both the calculated task completion time and the required number of charging stations decrease, which aligns with our expectations. In the figures, we observe that although the CEDAN algorithm uses fewer charging stations, its characteristic of forcibly adjusting the path sequence leads to longer task completion times. Additionally, we observe a proportional relationship between the total task time and the number of

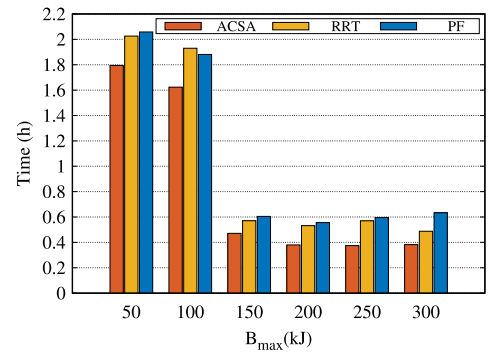


Fig. 8. Obstacle detour time.

charging stations. This is because, in our defined time function, the charging time at each charging station is significantly greater than the UAV's flight time between points, making the number of charging stations a dominant factor affecting task completion time. Furthermore, Fig. 7(a) shows that, compared with other algorithms, our proposed algorithm achieves shorter and more stable total task completion times.

F. Impact of Number of Grid Cell Size l

Our ACSA algorithm reduces the UAV flight distance by 62.92%, 65.54%, 69.31%, 15.95%, and 24.10% compared to

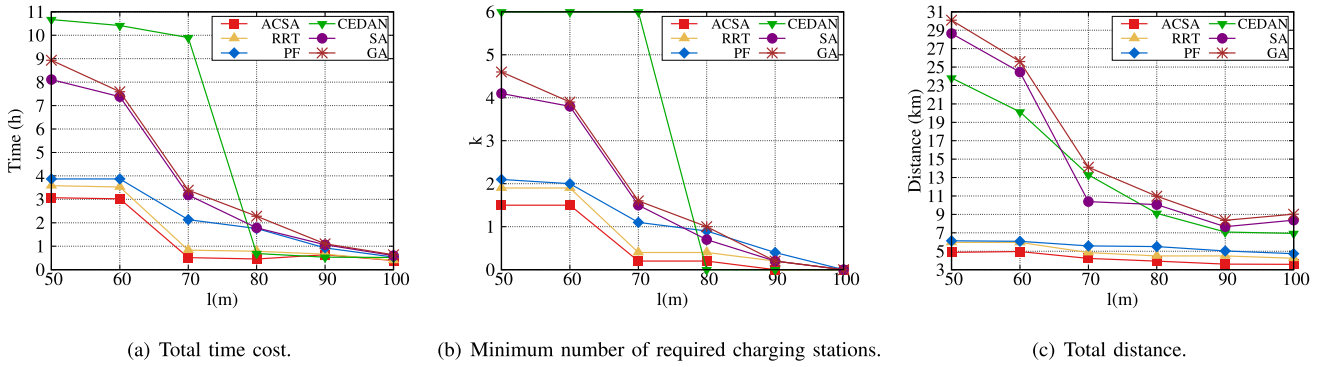


Fig. 9. Performance of different algorithms by varying the value of l from 50 m to 100 m.

CEDAN, SA, GA, RRT and PF, respectively, while also reducing the task completion time by 46.08%, 59.14%, 61.96%, 19.12%, and 41.88%, respectively.

Next, we compare the performance under different grid sizes l . In this simulation, we fix the UAV's maximum battery capacity $B_{\max}$ at 150 kJ and the maximum number of charging stations deployed in the scenario K at 6.

As shown in Fig. 9, we compare the impact of grid size variations on the total task completion time, the minimum number of charging stations required to be added to the trajectory, and the total path length. From Fig. 9(a), it can be seen that as the grid edge length increases, the total task completion time calculated by the six algorithms gradually decreases. At the same time, our proposed algorithm consistently achieves shorter completion times compared with the other five algorithms. This is also reflected in Fig. 9(b), where the number of charging stations required by our algorithm is fewer, which is proportional to the total time cost, further validating its effectiveness and correctness. This is because, as the grid edge length increases, the number of hovering positions M in the entire target area decreases, resulting in fewer nodes for the UAV to traverse. The reduction in nodes plays a significant role in lowering the total trajectory time, which aligns with our logical expectations. Fig. 9(c) further demonstrates that as the grid edge length increases, the total path length calculated by the six algorithms also gradually decreases. And our proposed algorithm exhibits significant advantages in both total path length and detour efficiency, whether compared with path optimization algorithms or obstacle-avoidance algorithms.

G. Impact of Number of Obstacles Q

Our ACSA algorithm reduces the UAV flight distance by 40.05%, 47.24%, 51.76%, 9.67%, and 17.42% compared with CEDAN, SA, GA, RRT, and PF respectively, while also reducing the task completion time by 29.86%, 36.28%, 40.56%, 6.50%, and 11.79%.

Additionally, to investigate the impact of the number of obstacles on the experimental results, we tested the outcomes of the algorithms as the number of obstacles increased from 0 to 10, as shown in Fig. 10. To highlight the effect of obstacle numbers, we fixed the UAV battery capacity at a sufficiently large value to

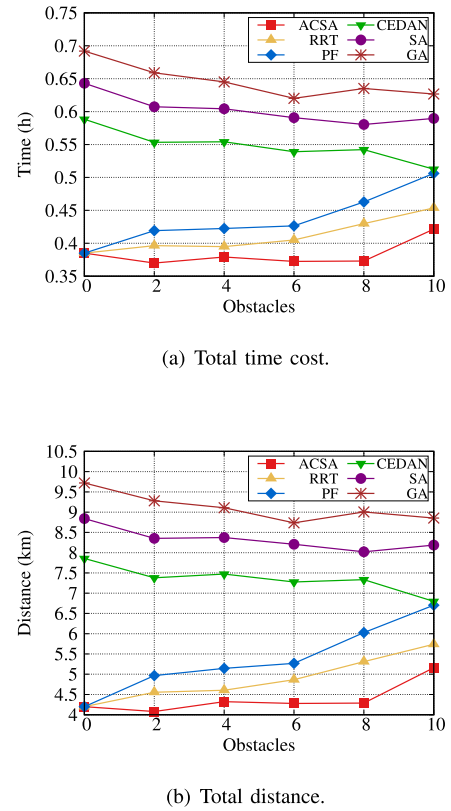


Fig. 10. Performance of different algorithms by varying the value of Q from 0 to 10.

ensure adequate power. Under these conditions, all algorithms required zero charging stations.

We compared the total completion time and total path length calculated by each algorithm, as shown in Fig. 10(a) and (b), respectively. It can be observed that the two metrics exhibit a positive correlation. This is because the total time is calculated as the sum of charging time and travel time. When the charging time is constant, travel time accounts for a larger proportion, making the total completion time proportional to the travel distance. The experimental results align well with our theoretical analysis.

Moreover, as the number of obstacles increases from 0 to 10, the gap between the two obstacle-avoidance algorithms

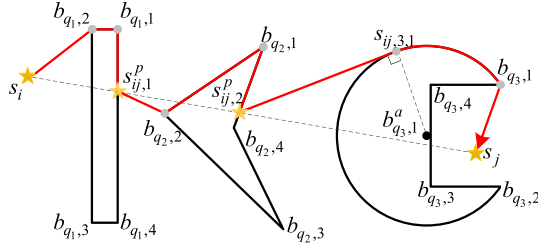


Fig. 11. Detour with Multi-Shaped Obstacles.

and the proposed algorithm gradually widens, and all three algorithms exhibit an upward trend. On the other hand, the remaining three path-planning algorithms inherently calculate longer path lengths, so the increase in path length due to obstacle avoidance has a relatively smaller impact on the overall results. Regardless of whether compared with path-planning algorithms or obstacle-avoidance algorithms, our proposed algorithm consistently produces superior and more stable solutions.

VII. DISCUSSION

Although the circumscribed cylinder approach for obstacle approximation proposed in this paper is applicable in most real-world scenarios, the UAV's hovering points may lie inside the circumscribed cylindrical structure of the obstacles in some cases, such as:

- 1) Obstacles in the form of long strips, such as buildings with a significant difference in length and width.
- 2) Obstacles with non-convex shapes.

In such cases, for obstacles of different shapes, we still represent the obstacle set as $\mathcal{O} = \{b_1, \dots, b_Q\}$. However, unlike the previous definition, each obstacle b_i is defined as a set containing vertex information (coordinates) and arc information (circle center coordinates and radii):

$$b_i = \{b_{i,1}, \dots, b_{i,Q_i^v}, \langle b_{i,1}^a, r_i \rangle, \dots, \langle b_{i,Q_i^a}^a, r_i \rangle\}, \quad (58)$$

where Q_i^v and Q_i^a are the number of vertices and arcs of b_i , respectively, and $b_{i,k}$ and $b_{i,k}^a$ represent coordinates of the vertex and the circle center corresponding to the arc of b_i , as shown in Fig. 11.

When designing a new obstacle avoidance algorithm, we retain the core strategy but substitute tangent points with vertices for straight-line segments. Specifically, the UAV flies from the current position towards the vertex (for a straight edge) or the tangent point of the nearest obstacle, follows the obstacle boundary, and exits the current obstacle where the line connecting two consecutive hovering points intersects the obstacle. This process repeats until there is no obstruction along the line connecting the current position and the ending hovering point; it then flies directly to the ending hovering in a straight line.

Let's take Fig. 11 as a toy example. The UAV needs to fly from s_i to s_j , but 3 obstacles blocks the trajectory. First, the current position is s_i and the nearest obstacle is b_{q1} . The algorithm calculates the distances $L_{s_i, b_{q1,1}}$, $L_{s_i, b_{q1,2}}$, $L_{s_i, b_{q1,3}}$, and $L_{s_i, b_{q1,4}}$, and obtains the minimum distance $L_{s_i, b_{q1,2}}$. Thus,

the UAV flies to $b_{q1,2}$. Then, the algorithm calculates which detour to the intersection point ($s_{ij,1}^p$) of b_{q1} and line $s_i s_j$ is the shortest, comparing the distances of paths $b_{q1,2} \rightarrow b_{q1,1} \rightarrow s_{ij,1}^p$ and $b_{q1,2} \rightarrow b_{q1,3} \rightarrow b_{q1,4} \rightarrow s_{ij,1}^p$. Obviously, the former path is shorter, so the UAV flies along it to $s_{ij,1}^p$. Next, similarly, the algorithm selects the nearest vertex $b_{q2,2}$ of b_{q2} and the shortest detour $b_{q2,2} \rightarrow b_{q2,1} \rightarrow s_{ij,2}^p$. Afterwards, for obstacle b_{q3} , the algorithm calculates the distances from $s_{ij,2}^p$ to all the vertices and the tangent points of the arc, and chooses the nearest point $s_{ij,3,1}$. Finally, the algorithm calculates the shortest detour to s_j , where the point at which the UAV leaves the last obstacle b_{q3} is a vertex or a tangent point of b_{q3} . So, the algorithm obtains the path $s_{ij,3,1} \rightarrow b_{q3,1} \rightarrow s_j$. And the final path has been obtained.

Note that this algorithm may not always obtain the shortest path. For example, in Fig. 11, the trajectory would be shorter if the UAV flew directly from $b_{q1,1}$ to $b_{q2,1}$ and from $b_{q2,1}$ to $s_{ij,3,1}$. An improvement would be to leave obstacles at the vertices or tangent points and then decide the next flying position. However, this may cause significant computational overhead when calculating the shortest path.

VIII. CONCLUSION

In this paper, we solve the Practical Optimizing UAV Trajectory in Wireless Charging Networks (POTWCN) problem in the presence of obstacles, to overcome the energy limitations encountered during UAV missions. We prove that this problem is NP-hard. The key technical depth lies in the design of an effective obstacle-avoidance trajectory with performance bound. We decompose the problem into two subproblems. We first design a geometry-based obstacle avoidance algorithm to complete the trajectory planning task with detour performance bound $3\pi/4$. Second, we propose a $(1 - \frac{1}{e})$ -approximation algorithm to balance the task completion time and the number of visited charging stations, leveraging the properties of the submodular function, and address the energy constraint issue by dynamically adjusting the number of charging stations. Experimental results demonstrate that our proposed algorithm is stable, and it can effectively reduce both the path length and the task completion time.

In the future, our goal is to consider the collaborative application of multiple UAVs in complex network environments.

REFERENCES

- [1] A. Al-Hourani, K. Sithamparamanathan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.
- [2] Marketsand markets. 2022. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/unmanned-aerial-vehicles-uav-market-662.html>
- [3] SF's drones. 2023. [Online]. Available: <https://piw-mr-web.inn.sf-express.com/>
- [4] PowerLight technologies. 2023. [Online]. Available: <https://powerlighttech.com/autonomous-vehicles.html>
- [5] J. Fingas, "Huawei wants to use lasers to charge drones, cell towers," 2017. [Online]. Available: <https://www.engadget.com/2017-02-27-huawei-drone-charging-cell-towers.html>
- [6] H. Pan, Y. Liu, G. Sun, J. Fan, S. Liang, and C. Yuen, "Joint power and 3D trajectory optimization for UAV-enabled wireless powered communication networks with obstacles," *IEEE Trans. Commun.*, vol. 71, no. 4, pp. 2364–2380, Apr. 2023.

- [7] X. Dai, Z. Xiao, H. Jiang, and J. C. S. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.
- [8] J. Xu, K. Ota, and M. Dong, "Aerial edge computing: Flying attitude-aware collaboration for multi-UAV," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5706–5718, Oct. 2023.
- [9] Z. Song, K. Chin, C. Yang, and M. Ros, "Methods to assign UAVs for K-coverage and recharging in IoT networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2504–2519, Apr. 2024.
- [10] M. Lahmeri, M. A. Kishk, and M. Alouini, "Laser-powered UAVs for wireless communication coverage: A large-scale deployment strategy," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 518–533, Jan. 2023.
- [11] P. Du, F. Xie, S. Chen, and X. Zhang, "Time-constrained UAV-aided data collection for IoT networks with energy harvesting," in *Proc. IEEE Conf. Comput. Commun. Workshop*, Hoboken, NJ, USA, 2023, pp. 1–6.
- [12] X. Hu, K. Wong, and Y. Zhang, "Wireless-powered edge computing with cooperative UAV: Task, time scheduling and trajectory design," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 8083–8098, Dec. 2020.
- [13] S. Aggarwal and N. Kumar, "Path planning techniques for unmanned aerial vehicles: A review, solutions, and challenges," *Comput. Commun.*, vol. 149, pp. 270–299, 2020.
- [14] W. Jiang, Y. Lyu, Y. Li, Y. Guo, and W. Zhang, "UAV path planning and collision avoidance in 3D environments based on pomdp and improved grey wolf optimizer," *Aerosp. Sci. Technol.*, vol. 121, 2022, Art. no. 107314.
- [15] N. Christofides, "Worst-case analysis of a new heuristic for the travelling salesman problem," *Oper. Res. Forum*, vol. 3, no. 1, 2022, Art. no. 10.
- [16] H. Gong, B. Huang, and B. Jia, "Energy-efficient 3-D UAV ground node accessing using the minimum number of UAVs," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12046–12060, Dec. 2024.
- [17] K. Wang, X. Zhang, L. Duan, and J. Tie, "Multi-UAV cooperative trajectory for servicing dynamic demands and charging battery," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1599–1614, Mar. 2023.
- [18] Y. Li et al., "Data collection maximization in IoT-sensor networks via an energy-constrained UAV," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 159–174, Jan. 2023.
- [19] C. Xiang et al., "Reusing delivery drones for urban crowdsensing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2972–2988, May 2023.
- [20] W. Xu et al., "Reward maximization for disaster zone monitoring with heterogeneous UAVs," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 890–903, Feb. 2024.
- [21] W. Xu et al., "Collect spatiotemporally correlated data in IoT networks with an energy-constrained UAV," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20486–20498, Jun. 2024.
- [22] Y. Rasekhipour, A. Khajepour, S. Chen, and B. Litkouhi, "A potential field-based model predictive path-planning controller for autonomous road vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 5, pp. 1255–1267, May 2017.
- [23] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. J. Teller, "Anytime motion planning using the RRT," in *Proc. IEEE Int. Conf. Robot. Automat.*, Shanghai, China, 2011, pp. 1478–1483.
- [24] L. Jaillet, J. Cortés, and T. Siméon, "Transition-based RRT for path planning in continuous cost spaces," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Nice, France, 2008, pp. 2145–2150.
- [25] Z. Tahir, A. H. Qureshi, Y. Ayaz, and R. Nawaz, "Potentially guided bidirectionalized RRT* for fast optimal path planning in cluttered environments," *Robot. Auton. Syst.*, vol. 108, pp. 13–27, 2018.
- [26] Y. Chen and L. Wang, "Adaptively dynamic RRT*-connect: Path planning for UAVs against dynamic obstacles," in *Proc. 7th Int. Conf. Automat. Control Robot. Eng.*, Xi'an, China, 2022, pp. 1–7.
- [27] J. W. Kim and S. K. Kim, "Fitness switching genetic algorithm for solving combinatorial optimization problems with rare feasible solutions," *J. Supercomput.*, vol. 72, no. 9, pp. 3549–3571, 2016.
- [28] S. Choueiry, M. Owayjan, H. Diab, and R. Achkar, "Mobile robot path planning using genetic algorithm in a static environment," in *Proc. 4th Int. Conf. Adv. Comput. Tools Eng. Appl.*, Beirut, Lebanon, 2019, pp. 1–6.
- [29] Y. V. Pehlivanoglu, "A new vibrational genetic algorithm enhanced with a Voronoi diagram for path planning of autonomous UAV," *Aerosp. Sci. Technol.*, vol. 16, no. 1, pp. 47–55, 2012.
- [30] M. da Silva Arantes, J. da Silva Arantes, C. F. M. Toledo, and B. C. Williams, "A hybrid multi-population genetic algorithm for UAV path planning," in *Proc. Genet. Evol. Comput. Conf.*, Denver, CO, USA, 2016, pp. 853–860.
- [31] Y. Wu, S. Wu, and X. Hu, "Cooperative path planning of UAVs & UGVs for a persistent surveillance task in urban environments," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4906–4919, Mar. 2021.
- [32] Y. Zhang, X. Liu, F. Bao, J. Chi, C. Zhang, and P. Liu, "Particle swarm optimization with adaptive learning strategy," *Knowl. Based Syst.*, vol. 196, 2020, Art. no. 105789.
- [33] M. Steinbrunn, G. Moerkotte, and A. Kemper, "Heuristic and randomized optimization for the join ordering problem," *VLDB J.*, vol. 6, no. 3, pp. 191–208, 1997.
- [34] Z. Yu, Z. Si, X. Li, D. Wang, and H. Song, "A novel hybrid particle swarm optimization algorithm for path planning of UAVs," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22547–22558, Nov. 2022.
- [35] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [36] J. Ouyang, Y. Che, J. Xu, and K. Wu, "Throughput maximization for laser-powered UAV wireless communication systems," in *Proc. IEEE Int. Conf. Commun. Workshops*, Kansas City, MO, USA, 2018, pp. 1–6.
- [37] D. Killinger, "Free space optics for laser communication through the air," *Opt. Photon. News*, vol. 13, no. 10, pp. 36–42, 2002.
- [38] H. Le, "A PTAS for subset TSP in minor-free graphs," in *Proc. ACM-SIAM Symp. Discrete Algorithms*, Salt Lake City, UT, USA, 2020, pp. 2279–2298.
- [39] H. Zhang and Y. Vorobeychik, "Submodular optimization with routing constraints," in *Proc. 13th AAAI Conf. Artif. Intell.*, Phoenix, Arizona, USA, 2016, pp. 819–826.
- [40] A. Bera, S. Misra, C. Chatterjee, and S. Mao, "CEDAN: Cost-effective data aggregation for UAV-enabled IoT networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5053–5063, Sep. 2023.



Yundi Wang received the BS degree from the School of Computer Science and Information, Anhui Normal University, in 2023. She is currently working toward the MS degree with the School of Computer Science and Technology, Soochow University. Her primary research interests lie in the fields of UAV networks and wireless charging.



Xiaoyu Wang received the BS degree in the School of Computer Science and Technology from Soochow University, Suzhou, Jiangsu, China, in 2016, and the PhD degree in the Department of Computer Science and Technology in Nanjing University, Nanjing, Jiangsu, China, in 2021. Her research interests focus on wireless charging and data mining. She is now an associate professor in School of Computer Science and Technology in Soochow University.



He Huang (Senior Member, IEEE) received the PhD degree from the School of Computer Science and Technology, University of Science and Technology of China (USTC), China, in 2011. He is currently a professor with the School of Computer Science and Technology, Soochow University, China. From 2019 to 2020, he was a visiting research scholar with Florida University, Gainesville. He has authored more than 100 papers in related international conference proceedings and journals. His current research interests include traffic measurement, computer networks, and algorithmic game theory. He is a member of the Association for Computing Machinery (ACM). He received the best paper awards from Bigcom 2016, IEEE MSN 2018, and Bigcom 2018. He has served as the Technical Program Committee member of several conferences, including IEEE INFOCOM, IEEE MASS, IEEE ICC, and IEEE Globecom.



Haipeng Dai (Senior Member, IEEE) received the BS degree in the Department of Electronic Engineering from Shanghai Jiao Tong University, Shanghai, China, in 2010, and the PhD degree in the Department of Computer Science and Technology in Nanjing University, Nanjing, China, in 2014. His research interests are mainly in the areas of wireless charging, mobile computing, and data mining. He is an associate professor in School of Computer Science in Nanjing University. He has authored more than 200 papers in many prestigious conferences and journals such as USENIX NSDI, ACM UbiComp, IEEE INFOCOM, USENIX ATC, ACM EuroSys, ACM SIGMOD, *Proceedings of the VLDB Endowment*, IEEE ICDE, ACM SIGMETRICS, ACM MobiSys, ACM MobiHoc, IEEE ICNP, IEEE IPSN, *IEEE Transactions on Mobile Computing*, *IEEE Journal on Selected Areas in Communications*, *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Parallel and Distributed Systems*, and IEEE TOSN. He is an IEEE senior member and ACM senior member. He serves/ed as the leading program chair of IEEE ISPA'22-23, the Co-Vice Program Chair of IEEE HPCC'21, Track Chair of the ICCCN'19 and ICPADS'21. He served as TPC member of international conferences such as INFOCOM, IJCAI, SC, VLDB, SIGKDD, MobiHoc, and ICNP. He received Best Paper Award from IEEE ICNP'15, Best Paper Award Runner-up from IEEE SECON'18, Best Paper Award Candidate from IEEE INFOCOM'17, Best Paper Award from IEEE HPCC'22, Best Paper Award from WASA'22, and Distinguished Paper Award from ACM UbiComp'22.